Docker is an open-source platform that **allows developers to automate the deployment, scaling, and management of applications inside lightweight, portable containers**. A container is **an isolated, self-contained environment** that **includes everything an application needs to run: code, libraries, dependencies, and system tools**.

Here's an overview of Docker's key concepts and components:

1. Containers

- **What are Containers?** Containers are a way to package an application and its dependencies in a consistent environment, regardless of the host system. They ensure that an application works the same way in any environment (development, staging, production) because the environment is containerized.
- **Isolation and Portability** Containers are isolated from the host system, but share the same OS kernel, making them lightweight compared to traditional virtual machines (VMs). This isolation means they won't interfere with other applications running on the same system. Containers can run on any system that has Docker installed, making them highly portable.

ELITHNEX
Shaping Careers, Building Futures

2. Dockerfile

- A `Dockerfile` is a script that contains a series of instructions on how to build a Docker image. It specifies things like:
  - Which base image to use
  - How to copy your application code into the image
  - How to install dependencies
  - What commands to run when the container starts

Example Dockerfile:

```
FROM node:14

COPY . .
RUN npm install
EXPOSE 3000
CMD ["npm", "start"]
```

3. Images

- **What is an Image?** A Docker image is a read-only template that contains the code, runtime, libraries, environment variables, and configuration files needed to run an application. Images are used to create containers. Docker images can be built from a `Dockerfile`, which contains a set of instructions on how to assemble the image.
- **Base Images and Custom Images** You can start with a base image (like Ubuntu, Alpine Linux, or Node.js) and then layer your application on top. Docker Hub is a central repository where you can find pre-built images for various programming languages and services.

4. Docker Engine

- **What is Docker Engine?** Docker Engine is the core component that runs containers. It is a client-server application with three major components:
    - **Docker Daemon (dockerd)**: The background service that manages containers, images, networks, and storage volumes.
      Note: dockerd is not being used now a days as it's obsolete. Now we are using **Containerd**
    - **Docker CLI (docker)**: The command-line interface that allows users to interact with the Docker Daemon.
    - **REST API**: Allows external programs to interact with Docker and manage containers.

5. Docker Compose

- Docker Compose is a tool for defining and running multi-container Docker applications. With a simple YAML file (`docker-compose.yml`), you can configure services (e.g., web servers, databases) and their relationships, networks, and volumes.

Example `docker-compose.yml`:

```
yaml

version: '3'
services:
  web:
    image: nginx
    ports:
      - "8080:80"
  db:
    image: sql
    environment:
```

PATHNEX
Shaping Careers, Building Futures

6. Docker Registry

- A Docker registry is where Docker images are stored. Docker Hub is the most popular public registry, but organizations can also use private registries. Registries allow you to push and pull images to and from remote repositories.

7. Networking

- **Network Isolation** Docker containers are isolated by default, and you can control how they communicate with each other and with the outside world. Docker networks enable this isolation and define how containers can communicate.
- **Default Network Types**:
    - **Bridge**: Default network for standalone containers.
    - **Host**: The container shares the host's networking stack.

    - **Overlay**: Used for multi-host networking in Swarm mode (Docker's orchestration tool).
    - **None**: No network interface for the container.

8. Volumes

- Volumes are used for persistent storage in Docker containers. By default, all data in a container is ephemeral (i.e., lost when the container is removed), but volumes allow you to persist data across container restarts or even container removals. Volumes can also be shared between containers.

9. Docker Swarm and Kubernetes

- **Docker Swarm** is Docker's native orchestration tool for managing clusters of Docker hosts (or nodes). It allows you to deploy, manage, and scale applications across a cluster of machines.
- **Kubernetes** is a more powerful container orchestration platform used to automate deployment, scaling, and operations of application containers. Kubernetes can be used with Docker (although Docker is being replaced as the default container runtime in some environments).

10. Benefits of Docker

- **Portability**: Containers run the same regardless of the host system.
- **Efficiency**: Containers are lightweight and start up quickly.
- **Isolation**: Applications and services run in isolated environments, reducing dependency conflicts.
- **Version Control**: Docker images are versioned, so you can easily roll back to previous versions.
- **CI/CD Integration**: Docker integrates well with continuous integration and deployment pipelines.

PATHNEX
Shaping Careers, Building Futures

11. Common Docker Commands

- `docker build`: Builds a Docker image from a Dockerfile.
- `docker run`: Runs a container from an image.
- `docker ps`: Lists running containers.
- `docker stop`: Stops a running container.
- `docker rm`: Removes a stopped container.
- `docker pull`: Pulls an image from a registry.
- `docker push`: Pushes an image to a registry.

12. Use Cases for Docker

- **Microservices**: Docker is often used in microservice architectures, where each service runs in its own container.
- **DevOps & CI/CD**: Docker containers can be easily integrated into CI/CD pipelines to provide consistent environments for testing and production.

- **Cloud Deployment**: Docker is widely used in cloud environments for application deployment and scaling.
- **Environment Consistency**: Docker helps ensure that the environment for development, testing, and production are consistent.

Conclusion

Docker simplifies the process of developing, shipping, and running applications by using containers, making it easier to manage dependencies, scale applications, and maintain consistent environments across different stages of the software lifecycle.

Sequence of commonly used Docker commands.

Note: bash in the starting on the commands tell that these run on bash terminal(commands). Yaml shows that is a yaml script.

To install Docker on Ubuntu, you can run:

```
bash

sudo apt update
sudo apt install docker.io
sudo systemctl enable --now docker
```

Once Docker is installed, verify the installation:

```
bash

docker --version
```

Check the status of Docker service:

```
bash

sudo systemctl status docker
```

To pull the Nginx image from Docker Hub:

```
bash

docker pull nginx
```

List the Docker images on your system:

```
bash

docker images
```

Run the Nginx container interactively:

PATHNEX
Shaping Careers, Building Futures

```
bash

docker run -it nginx /bin/bash
```

Run the Nginx container in detached mode:

```
Bash
docker run -d --name nginx-container nginx
```

List all running containers:

```
bash

docker ps
```

List all containers, including stopped ones:

```
bash

docker ps -a
```

Stop the running Nginx container:

```
bash

docker stop nginx-container
```

Start the stopped Nginx container:

```
bash

docker start nginx-container
```

Remove the Nginx container (ensure it's stopped first):

```
bash
docker rm nginx-container
```

Remove the Nginx image:

```
bash

docker rmi nginx
```

Build a custom image from a `Dockerfile` (if you have one, for example):

```
bash

docker build -t nginx-image .
```

This specifies a tag (-t) for the image, in this case, nginx-image. Tags are useful for naming and versioning your images. In this example, the image will be named nginx-image.

Tag your custom image:

PATHNEX
Shaping Careers, Building Futures

```
bash

docker tag nginx-image repo/nginx-image:v1
```

Push your custom image to Docker Hub (after logging in):

```
bash

docker login
docker push repo/nginx-image:v1
```

Inspect the Nginx container:

```
bash

docker inspect nginx-container
```

View logs of the Nginx container:

```
bash

docker logs nginx-container
```

Run the Nginx container with volume mapping (to map a local directory to the container's Nginx directory):

```
bash

docker run -d -v /path/to/host/directory:/usr/share/nginx/html -p 8080:80
nginx
```

Run the Nginx container with port mapping (map port 8080 on your host to port 80 in the container):

```
bash

docker run -d -p 8080:80 nginx
```

To interact with a running Nginx container:

```
bash

docker exec -it nginx-container /bin/bash
```

PATHNEX
Shaping Careers, Building Futures

Command	Description
`docker system prune`	Removes stopped containers, unused networks, dangling images, and unused volumes.
`docker system prune -a`	More aggressive — removes **all unused images**, not just dangling ones.
`docker container prune`	Removes **stopped containers**.
`docker image prune`	Removes **dangling images** (images not tagged or in use).
`docker image prune -a`	Removes **all unused images**, including non-dangling ones.
`docker volume prune`	Removes **unused volumes**.
`docker network prune`	Removes **unused networks**.
`docker builder prune`	Removes **build cache** used during image builds.

PATHNEX
Shaping Careers, Building Futures